# Build Your Own Mini Scripting Engine

*Take-Home Group Project — Advanced Object-Oriented Programming in Java*

Sitare University | Groups of 3 | 12.5 marks

## What Is This Project?

Every time you write a line of Java and it runs, something remarkable happens behind the scenes — a program reads your code, understands its structure, and executes it. Have you ever wondered how that actually works? This project gives you a chance to find out firsthand.

You are going to build a working interpreter for a small made-up programming language — entirely in pure Java. When you are done, a user will be able to write code in your language, run your program, and see real output on screen. That is not a toy exercise. That is genuinely impressive software, and you will understand how it works at every level because you built it yourself.

The skills you develop here — designing class hierarchies, thinking carefully about how objects relate to each other, separating responsibilities cleanly across classes — are exactly the skills that make a strong software engineer. Every real programming language you have ever used was built by people who thought through the same kinds of problems you will face in this project.

This will be challenging. There will be moments where something does not work and you are not sure why. That feeling is normal — it is what real software development feels like. Push through it as a team, and the satisfaction of seeing your interpreter correctly run a program for the first time will make it worth it.

> *You do not need any background in compilers or programming language theory. Everything you need is explained step by step in this document. Read it fully before writing a single line of code.*

## The Four Languages

There are four toy languages. I will assign one to your group. They all have the same difficulty and the same underlying structure — the only difference is the syntax style.

**ZARA** *Zero-ceremony Arithmetic and Reasoning Assembler*
*Clean and minimal. If Python were even simpler.*

**How it looks**

```
set x = 10
set y = 20
```

**What you will build**

- Tokenizer recognising: set, show, when, loop

```
set z = x + y * 2
show z

when z > 30:
    show "big number"

loop 3:
    show "hello"
```

- Expression tree for arithmetic and comparisons
- Variable store to track values during execution
- Instruction runner that executes each instruction
- CLI entry point to run .zara source files

## BLOOP *Beginner-Level Object-Oriented Program*

*Reads almost like plain English sentences.*

**How it looks**

```
put 10 into x
put 20 into y
put x + y * 2 into z
print z

if z > 30 then:
    print "big number"

repeat 3 times:
    print "hello"
```

**What you will build**

- Tokenizer recognising: put, into, print, if, then, repeat, times
- Expression tree for arithmetic and comparisons
- Variable store to track values during execution
- Instruction runner that executes each instruction
- CLI entry point to run .bloop source files

## CALC *Concise Algorithmic Language for Computation*

*Math-style notation. Symbols over words.*

**How it looks**

```
x := 10
y := 20
z := x + y * 2
>> z

? z > 30 =>
    >> "big number"

@ 3 =>
    >> "hello"
```

**What you will build**

- Tokenizer recognising: `:=` `>>` `?` `@` `=>`
- Expression tree for arithmetic and comparisons
- Variable store to track values during execution
- Instruction runner that executes each instruction
- CLI entry point to run .calc source files

## SPEEK *Simple Plain English Execution Kit*

*Reads like you are dictating instructions out loud.*

**How it looks**

```
let x be 10
let y be 20
let z be x + y * 2
say z

if z is greater than 30 then
    say "big number"

repeat 3 times
    say "hello"
```

**What you will build**

- Tokenizer recognising: let, be, say, if, is greater than, then, repeat, times
- Expression tree for arithmetic and comparisons
- Variable store to track values during execution
- Instruction runner that executes each instruction
- CLI entry point to run .speek source files

## How Your Interpreter Works

Your interpreter is a pipeline of three steps. Each step takes the output of the previous one and hands its result to the next.

| Step | Name | What it does |
|---|---|---|
| Step 1 | Tokenizer | Reads source code character by character and breaks it into a flat list of labelled pieces called tokens — like splitting a sentence into individual words. |
| Step 2 | Parser | Reads the token list and builds a tree of objects that captures the structure of the program, including the correct order of operations. |
| Step 3 | Evaluator | Walks the tree from the bottom up, executing each instruction and producing the final output. |

### Why a Tree?

Arithmetic has structure. In the expression x + y * 2, multiplication must happen before addition. A flat list of tokens cannot capture this — but a tree can. The multiply node sits deeper in the tree than the add node, so it gets evaluated first. You do not need any special tricks: the shape of the tree handles operator precedence automatically.

```
      Add
     /   \
    x    Multiply
         /    \
        y      2
```

## Class and Interface Structures

Below is the skeleton of every class and interface you must implement. The method bodies are intentionally empty — working out what goes inside each one is the core of this project. Read the comments carefully; they describe exactly what each method must do.

You are free to add helper methods and extra classes as you see fit. You must not remove any of the classes or methods listed here.

> **On keywords:** All class names below apply to every language. The only thing specific to your language is which keywords appear in your TokenType enum. Refer to your language's box above for the exact list.

### 1. TokenType and Token

TokenType is an enum listing every kind of token your language can produce. Token is a simple immutable object holding one piece of source code — its type, its raw text, and the line it came from.

```
public enum TokenType {
    // Add one entry for each kind of token your language has.
    // Every language needs at minimum: NUMBER, STRING, IDENTIFIER,
    // arithmetic operators, comparison operators, your keywords, NEWLINE,
EOF.
}
```

```
public class Token {
    // Fields: the type, the raw text value, and the line number.
    // All fields must be set in the constructor and never changed after
that.

    public Token(TokenType type, String value, int line) {
        // Store the three fields.
    }

    // Provide getters for each field. No setters.
}
```

### 2. Expression Interface and Node Classes

Every expression in your language — a number, a variable name, a calculation — is represented as a node object. All node classes implement the Expression interface, which has a single method: evaluate.

```
public interface Expression {
    // Evaluate this expression using the current variable store.
    // Returns either a Double (for numbers) or a String (for text).
    Object evaluate(Environment env);
}
```

### NumberNode

Represents a literal number in the source code, such as 42 or 3.14.

```
public class NumberNode implements Expression {
    // Store the numeric value as a double.
    // Set it in the constructor. No setters.

    @Override
    public Object evaluate(Environment env) {
        // Return the stored number.
    }
}
```

### StringNode

Represents a literal string in the source code, such as "hello".

```
public class StringNode implements Expression {
    // Store the string value.
    // Set it in the constructor. No setters.

    @Override
    public Object evaluate(Environment env) {
        // Return the stored string.
    }
}
```

### VariableNode

Represents a variable reference such as x or total. When evaluated, it looks up the variable's current value in the Environment.

```
public class VariableNode implements Expression {
    // Store the variable name as a String.
    // Set it in the constructor. No setters.

    @Override
    public Object evaluate(Environment env) {
        // Ask the Environment for the current value of this variable.
        // Return whatever the Environment gives back.
    }
}
```

### BinaryOpNode

Represents an operation between two expressions, such as x + y * 2. It holds a left expression, an operator symbol, and a right expression. When evaluated, it evaluates both sides first and then applies the operator.

```
public class BinaryOpNode implements Expression {
    // Store: left expression, operator symbol (e.g. "+", "-", ">"), right
expression.
    // Set all three in the constructor. No setters.

    @Override
    public Object evaluate(Environment env) {
        // 1. Evaluate the left expression.
        // 2. Evaluate the right expression.
        // 3. Apply the operator to the two results and return the answer.
        //    Arithmetic operators return a Double.
        //    Comparison operators (>, <, ==) return a Boolean.
    }
}
```

## 3. Environment

The Environment is the variable store — a simple map from variable names to their current values. Every instruction shares one Environment instance during execution.

```
public class Environment {
    // Use a Map<String, Object> to store variable names and their values.

    public void set(String name, Object value) {
        // Store or update the value for the given variable name.
    }

    public Object get(String name) {
        // Return the current value of the variable.
        // If the variable has not been defined, throw a RuntimeException
        // with a helpful message such as "Variable not defined: x".
    }
}
```

## 4. Instruction Interface and Classes

An instruction is one complete action your language can perform. All instruction classes implement the Instruction interface.

```
public interface Instruction {
    // Execute this instruction, reading and writing variables via the
Environment.
    void execute(Environment env);
}
```

### AssignInstruction

Handles assignment — for example `set x = 10 + 5` in ZARA or `x := 10 + 5` in CALC.

```
public class AssignInstruction implements Instruction {
    // Store: the variable name and the expression whose value will be
assigned.

    @Override
    public void execute(Environment env) {
        // Evaluate the expression, then store the result in the
Environment.
    }
}
```

### PrintInstruction

Handles printing — for example `show x` in ZARA or `>> x` in CALC.

```
public class PrintInstruction implements Instruction {
    // Store: the expression to print.

    @Override
    public void execute(Environment env) {
        // Evaluate the expression and print the result to standard output.
    }
}
```

### IfInstruction

Handles a conditional block. If the condition evaluates to true, the body instructions are executed; otherwise they are skipped.

```
public class IfInstruction implements Instruction {
    // Store: the condition expression and a List<Instruction> for the
body.

    @Override
    public void execute(Environment env) {
        // Evaluate the condition.
        // If the result is true, execute each instruction in the body.
    }
}
```

### RepeatInstruction

Handles a loop that runs its body a fixed number of times.

```
public class RepeatInstruction implements Instruction {
    // Store: the repeat count and a List<Instruction> for the body.

    @Override
    public void execute(Environment env) {
        // Execute all body instructions, repeated count times.
    }
}
```

## 5. Tokenizer

The Tokenizer reads the raw source code as a single String and produces a List<Token>. It walks through the characters one at a time, recognises patterns, and emits a Token for each one.

```
public class Tokenizer {
    // Store: the source string and a current position index.

    public Tokenizer(String source) {
        // Store the source.
    }

    public List<Token> tokenize() {
        // Walk through source character by character.
        // When you recognise a complete token, add it to a list.
        // At the end, add a token of type EOF.
        // Return the completed list.
    }
}
```

## 6. Parser

The Parser reads the List<Token> produced by the Tokenizer and builds a List<Instruction>. It looks at one token at a time and decides what kind of instruction or expression it is reading.

```
public class Parser {
    // Store: the token list and a current index tracking which token
    // you are looking at right now.

    public Parser(List<Token> tokens) {
        // Store the token list.
    }

    public List<Instruction> parse() {
        // Read through tokens and build the list of Instructions.
        // Return the completed list.
    }

    // You will need several private helper methods here.
    // A natural split for handling operator precedence:
    //   parseExpression() -- handles + and -
    //   parseTerm()       -- handles * and /
    //   parsePrimary()    -- handles a single number, string, or variable
    // parseExpression calls parseTerm; parseTerm calls parsePrimary.
    // This chain gives * and / higher priority than + and - automatically.
}
```

## 7. Interpreter

The Interpreter is the entry point that connects all three steps. Its run method accepts source code as a String and produces the program's output.

```
public class Interpreter {
```

```
    public void run(String sourceCode) {
        // Step 1: Pass sourceCode to a new Tokenizer and get the token
list.
        // Step 2: Pass the token list to a new Parser and get the
instruction list.
        // Step 3: Create a new Environment, then execute each instruction.
    }
}
```

Your main method should read a source file path from the command-line argument, read its contents into a String, and pass that String to Interpreter.run().

---

## Sample Programs

Your interpreter must produce the correct output for all four programs in your language. Samples for all four languages are shown below.

### ZARA

#### Program 1 — Arithmetic and variables

```
set x = 10
set y = 3
set result = x + y * 2
show result
```

Expected output:

```
16
```

#### Program 2 — String output

```
set name = "Sitare"
show name
show "Hello from ZARA"
```

Expected output:

```
Sitare
Hello from ZARA
```

#### Program 3 — Conditional

```
set score = 85
when score > 50:
    show "Pass"
```

Expected output:

```
Pass
```

### Program 4 — Loop

```
set i = 1
loop 4:
    show i
    set i = i + 1
```

Expected output:

```
1
2
3
4
```

## BLOOP

### Program 1 — Arithmetic and variables

```
put 10 into x
put 3 into y
put x + y * 2 into result
print result
```

Expected output:

```
16
```

### Program 2 — String output

```
put "Sitare" into name
print name
print "Hello from BLOOP"
```

Expected output:

```
Sitare
Hello from BLOOP
```

### Program 3 — Conditional

```
put 85 into score
if score > 50 then:
    print "Pass"
```

Expected output:

```
Pass
```

### Program 4 — Loop

```
put 1 into i
repeat 4 times:
    print i
    put i + 1 into i
```

Expected output:

```
1
2
3
4
```

## CALC

### Program 1 — Arithmetic and variables

```
x := 10
y := 3
result := x + y * 2
>> result
```

Expected output:

```
16
```

### Program 2 — String output

```
name := "Sitare"
>> name
>> "Hello from CALC"
```

Expected output:

```
Sitare
Hello from CALC
```

### Program 3 — Conditional

```
score := 85
? score > 50 =>
    >> "Pass"
```

Expected output:

```
Pass
```

### Program 4 — Loop

```
i := 1
@ 4 =>
    >> i
```

```
    i := i + 1
```

Expected output:

```
1
2
3
4
```

## SPEEK

### Program 1 — Arithmetic and variables

```
let x be 10
let y be 3
let result be x + y * 2
say result
```

Expected output:

```
16
```

### Program 2 — String output

```
let name be "Sitare"
say name
say "Hello from SPEEK"
```

Expected output:

```
Sitare
Hello from SPEEK
```

### Program 3 — Conditional

```
let score be 85
if score is greater than 50 then
    say "Pass"
```

Expected output:

```
Pass
```

### Program 4 — Loop

```
let i be 1
repeat 4 times
    say i
    let i be i + 1
```

Expected output:

```
1
```

```
2
3
4
```

### Extension Options (Optional, can implement any one if you want, whichever interests you more)

- Else block — handle both the true and false case of a condition
- While loop — a loop that keeps running as long as a condition is true
- String length — a built-in operation to get the number of characters in a string
- Nested blocks — allow if or loop instructions inside other if or loop blocks
- Helpful error messages — when something goes wrong, print a clear message with the line number instead of crashing
- Equality check — support == comparisons to test whether two values are equal

---

## Viva — Oral Examination

After the final submission every group attends a 15-minute viva. All three members must be present. Each member will be asked questions individually about the code your group submitted.

The viva is not a trick. If you wrote the code and understand it, you will do well. Typical questions include:

- "Walk me through what happens step by step when your interpreter runs one line of code."
- "Where exactly in your code does operator precedence get handled?"
- "What is the Environment class for? What would break if you removed it?"
- "I will write a line in your language on the board right now — tell me what your program does with it."
- "If you wanted to support a new operator, what would you change?"

Members who cannot answer questions about their own code will lose marks individually, even if the group submission is correct. Being able to explain and defend your design decisions under pressure is precisely what every software engineering interview tests. This is your practice for that.

## Tips

### Build and test one small thing at a time

Do not try to build the whole interpreter at once. First get the Tokenizer to correctly break up a single line. Then get the Parser to handle one instruction. Then execute it correctly. Each small working step gives you momentum and makes bugs much easier to find.

### Test with the simplest programs first

Before you try anything with loops or conditions, make sure a simple assign-then-print program works end to end. Complexity is much easier to add once the foundation is solid.

## Divide the work thoughtfully

A natural split is one member on the Tokenizer, one on the Parser, and one on the instruction and expression classes. But all three members must understand all three parts — the viva will ask any member about any section of the code.

## Read error messages carefully

When Java throws an exception, read the full message. It almost always tells you exactly what went wrong and on which line. Randomly changing code hoping it will fix itself rarely works and wastes time.

## On using AI tools

Use of AI tools is allowed for this project. But if you submit code you do not understand, the viva will make that obvious within the first minute. The most valuable way to use any tool is to ask it to explain a concept you are confused about — not to generate your solution. Your goal at the end of this project is to be able to sit in front of an interviewer and walk them through every decision your team made. That is a skill worth building now.

*Good luck — and enjoy the experience of building something that actually runs code.*